PROJECT

GetFit Daily

An AI calorie tracker: photograph a meal, get its calories and macros · Solo Developer · 2026

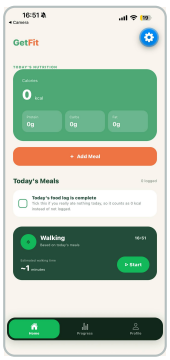
`github.com/pskpsk009/AiCalorieTracker`

Most calorie trackers make you search a food database and type in portions by hand, which is enough effort that people stop logging. GetFit Daily cuts logging down to one step: **take a picture of your plate**. A Gemini vision model returns each food item with its weight, calories, protein, carbs, and fat. You check it and save it. A weekly dashboard then puts what you ate next to how much you walked and your weight history, all from your own data.

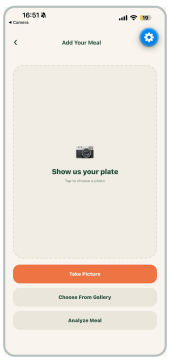
Expo SDK 57 · React Native 0.86 · TypeScript · Expo Router · Google Gemini · Supabase · Postgres + RLS · Deno Edge Functions · expo-image-picker · Jest

Size: about 3,200 lines of TypeScript · 6 Postgres tables with Row Level Security · versioned SQL migrations · 36 unit tests

Logging a Meal



Home: today's totals



Add meal: camera or gallery

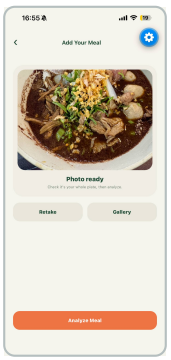
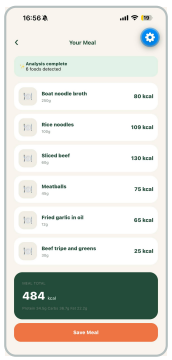


Photo ready to analyze



AI analysis: per-item breakdown



Saved meal on Home

1. Pick
`expo-image-picker` returns a photo from the camera or gallery.

2. Upload
Stored in a private, per-user `meal-images` folder; read back by a 1-hour signed URL.

3. Analyze
The `analyze-meal` Edge Function checks the JWT, then asks Gemini for JSON only.

4. Validate
Every number is checked, clamped, and rounded before it reaches Postgres.

5. Display
One `meals` row and its `meal_items` are saved; today's totals update.

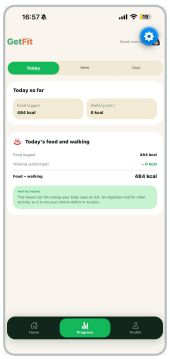
Home

- **Today's totals** for calories, protein, carbs, and fat, with every meal and its food items listed below.
- A **walking tracker** with start, pause, resume, and stop, and a live calorie-burn estimate labelled as an estimate.
- Expo Router with a bottom bar: **Home**, **Progress**, and **Profile**.

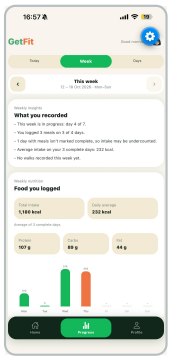
AI analysis

- Each detected item shows its **grams, calories, and macros**; the meal total is the sum of its items.
- The Edge Function **retries** rate limits (429) and outages (503) up to 3 times with backoff, then **falls back** to a second Gemini model.
- A bad value from the model fails with an error naming the item and field, instead of saving a wrong number.

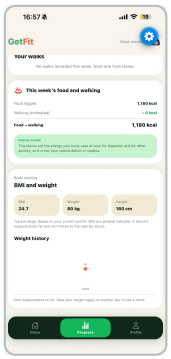
Progress & Profile



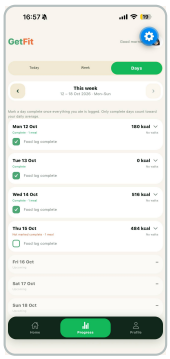
Progress: today



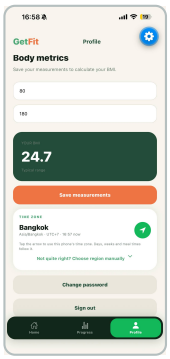
Weekly dashboard



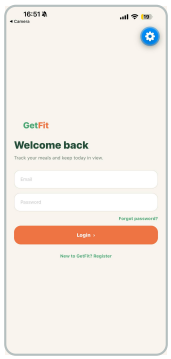
BMI and weight history



Day-by-day history



Profile and time zone



Sign in

Weekly dashboard

- Pick any **Monday–Sunday week** for totals, averages, macros, walking time, daily charts, and plain-language insights.
- **Tap a day** to see that day's meals and walks.
- **"Log complete" days**: a day with no meals shows as *not logged* unless you mark it complete, so "ate nothing" and "forgot to log" never look the same. Averages count only complete days.

Profile & account

- **Live BMI** and a **dated weight chart**, recorded by a database trigger whenever the profile weight changes.
- **Per-user time zone**, detected from the phone in one tap or chosen by region and city.
- Sign-up with **deep-link email confirmation** (`getfit://auth/callback`), plus forgot-password and change-password flows.

Key Engineering Decisions

AI Treat AI output as untrusted input

The function asks for JSON only and strips stray markdown fences. On the client each value is coerced to a number, checked with `Number.isFinite`, clamped to ≥ 0, and rounded before it is saved.

Data Keep a two-table write consistent

A meal is one `meals` row plus several `meal_items`. The Supabase JS client can't wrap both in a transaction, so if the items fail the parent is deleted (a compensating rollback). At scale this would move into a Postgres RPC.

Time Time zones done properly

The database stores UTC, but "today" and "this week" mean the user's local day. Each local day is turned into exact UTC bounds before querying. Tests walk every day of a year in 8 time zones, including 23- and 25-hour DST days and half-hour offsets.

Security Security in the database, not just the app

Row Level Security on every table, a private photo bucket with per-user folders and short-lived signed URLs, and the Edge Function verifies the caller's JWT before doing any paid work.

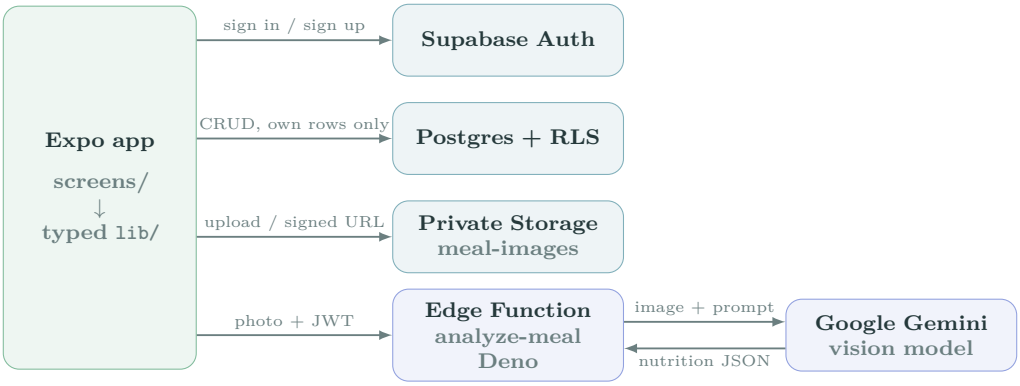
Honesty Don't show misleading health numbers

Burned calories are labelled as estimates, and food minus walking is shown as a partial figure, never as a deficit or surplus. Gaps are never filled with sample data.

Structure Keep logic out of screens

Route files are thin, UI lives in `screens/`, and all Supabase access goes through typed functions in `lib/`. The weekly aggregation is a pure function, so it is unit-tested directly.

Architecture & Data Model



Tables

- `profiles`: name, weight, height, time zone
- `weight_measurements`: written by trigger
- `food_day_confirmations`: "log complete" days
- `meals` → `meal_items`
- `walking_sessions`: duration and burn estimate

All foreign keys cascade on delete: removing a user removes their data, and removing a meal removes its items.

Testing

- **36 Jest unit tests** on the logic easiest to get subtly wrong.
- **Date ranges**: DST, half-hour offsets, week and year boundaries.
- **Weekly summary**: daily totals add up to weekly totals, duplicates count once, out-of-range rows are ignored.
- **Time zones** and **weight trend**: detection, validation, de-duplication by day.

What I learned & next steps

- **Never trust a model's output.** Prompting for structured JSON helps, but the app still has to validate every field, and handle rate limits and outages with retries and a fallback model.
- **Access control belongs in the database.** Row Level Security and private storage keep users' data apart even if the client has a bug.
- **Dates are harder than they look.** "This week" depends on the user's time zone, and DST days aren't 24 hours long; tests across a whole year made the date code trustworthy.
- **Missing data is not zero.** Separating "not logged" from "ate nothing" changed how every average is calculated.
- **Next:** let users edit detected items and grams before saving, move the meal save into a Postgres RPC, step counting with `expo-sensors`, estimated daily energy needs (Mifflin-St Jeor), and UI and end-to-end tests.

GetFit Daily's calorie and macro figures are AI estimates from a single photo. It is not a medical or dietary tool.